Parcial Segundo Corte — Sistemas Complejos

Nombre: Juan Camilo Gallardo Sánchez

Universidad: Universidad Sergio Arboleda

Programa: Ciencias de la Computación e Inteligencia Artificial

Curso: Sistemas Complejos

Fecha: 15 de abril de 2026

Punto 1: Complejidad de Internet mediante un Modelo Matemático

¿Qué es complejidad?

Antes de modelar Internet matemáticamente es necesario precisar qué entendemos por un sistema complejo. Un sistema es complejo cuando exhibe simultáneamente las siguientes propiedades:

1. Gran número de componentes que interactúan localmente, sin un control central que dicte el comportamiento global.

2. Interacciones no lineales y con retroalimentación, de modo que pequeños cambios locales pueden producir efectos globales desproporcionados.

3. Emergencia: aparecen propiedades macroscópicas (distribuciones, patrones, estructuras) que no están presentes en ningún componente por separado.

4. Auto-organización y robustez estructural frente a perturbaciones aleatorias, junto con fragilidad frente a perturbaciones dirigidas.

5. Dependencia sensible de la historia y de los detalles de las interacciones, lo que impide una descripción puramente reduccionista.

La complejidad, entonces, no es un atributo numérico sino un régimen de comportamiento: un sistema es complejo cuando su descripción global sólo puede obtenerse a través de sus interacciones.

Cómo el modelo de Internet realiza cada propiedad

El modelo matemático que se desarrolla a continuación (grafo con distribución de grado libre de escala y crecimiento con conexión preferencial al estilo Barabási–Albert) describe la complejidad de Internet anclando cada propiedad anterior a un elemento formal:

• Gran número de componentes que interactúan localmente los nodos del grafo G=(V,E) y sus aristas; ningún nodo conoce la topología global, sólo su vecindario.

• No linealidad y retroalimentación la regla de conexión preferencial P(i) k_i, que amplifica a los nodos ya conectados (efecto rich-gets-richer).

• Emergencia la distribución de grado P(k) k^(-γ) no es impuesta por ningún nodo; surge como propiedad global del ensamble.

• Auto-organización robusta y fragilidad dirigida la red sobrevive a la caída del 80% de nodos aleatorios, pero colapsa al retirar los 5–10 hubs principales. Dualidad característica de los sistemas complejos.

• Dependencia de la historia la topología actual es producto del orden de llegada de nodos y del valor k_i en cada paso t; no existe una fórmula cerrada independiente del proceso de crecimiento.

Por tanto, afirmar que Internet es complejo significa, desde este modelo, que su estructura y su comportamiento sólo son comprensibles como propiedades emergentes de una dinámica local de crecimiento con acoplamiento preferencial.

Modelado como Grafo

Internet se representa formalmente como un grafo dirigido $G=(V,E)$, donde V es el conjunto de nodos (routers, servidores, páginas web) y E el conjunto de aristas (enlaces físicos o lógicos entre ellos). El número de conexiones de un nodo *i* se denomina su **grado** k_i, y constituye la métrica fundamental de su relevancia dentro de la red.

En un modelo de red aleatoria clásico (Erdős–Rényi), cada par de nodos se conecta con probabilidad p independiente, generando una distribución de grado de Poisson:

$$P(k)=\binom{N-1}{k}p^k(1-p)^{N-1-k}\approx\frac{\langle k\rangle^k e^{-\langle k\rangle}}{k!}$$

Sin embargo, Internet **no sigue esta distribución**, lo que exige un modelo matemático diferente.

Redes Libres de Escala y Ley de Potencias

El análisis empírico de Internet demuestra que su distribución de grado obedece una **ley de potencias** (Barabási & Albert, 1999):

$$P(k)\sim k^{-\gamma},\ \ \gamma\in[2,3]$$

Esto implica que la red es **heterogénea**: la mayoría de nodos tienen pocas conexiones, mientras que unos pocos *hubs* (como Google, Amazon o Cloudflare) concentran miles de enlaces. Esta distribución de cola pesada es la firma matemática de un sistema complejo.

Modelo Barabási–Albert (BA)

El modelo BA explica el surgimiento de la ley de potencias mediante dos mecanismos acoplados:

1. Crecimiento: La red comienza con m_0 nodos. En cada paso de tiempo t, se añade un nuevo nodo con $m \leq m_0$ aristas.

2. Conexión preferencial (*preferential attachment):* La probabilidad de que el nuevo nodo se conecte al nodo existente i es proporcional a su grado actual:

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}$$

Este mecanismo produce el efecto *"rich gets richer"*: los nodos más conectados acumulan nuevos enlaces con mayor probabilidad. Tras t pasos, la distribución de grado converge a:

$$P(k) \sim 2m^2 \cdot k^{-3}$$

lo que reproduce el exponente $\gamma = 3$ observado empíricamente.

Métricas de Complejidad de Internet

La complejidad de Internet no se reduce a su distribución de grado. Se caracteriza además por las siguientes métricas estructurales:

Métrica	Definición Matemática	Valor típico en Internet
Coeficiente de clustering C	$C_i = \frac{2e_i}{k_i(k_i - 1)}$	Alto en nodos periféricos
Distancia media $\langle d \rangle$	$\langle d \rangle = \frac{1}{N(N-1)} \sum_{i \neq j} d(i,j)$	~19 clics entre páginas
Exponente γ	De $P(k) \sim k^{-\gamma}$	≈ 2.1 (WWW)
Diámetro D	Máxima distancia geodésica	Pequeño (*small world*)

El alto C junto con el pequeño $\langle d \rangle$ define la propiedad **small world**: agrupación local elevada con separación global reducida.

Robustez y Vulnerabilidad como Propiedad Emergente

La distribución de ley de potencias genera una dualidad de complejidad:

- **Robustez ante fallos aleatorios:** La probabilidad de que un nodo aleatorio sea un *hub* es $P(k > k_c) \sim k_c^{1-\gamma} \to 0$. La red permanece conectada incluso si se eliminan hasta el 80% de los nodos aleatoriamente.
- **Vulnerabilidad ante ataques dirigidos:** Eliminar los 5–10 *hubs* principales fragmenta la red casi de inmediato, ya que su eliminación destruye las rutas más cortas del sistema.

Esta dualidad es una **propiedad emergente** del sistema complejo: no puede inferirse estudiando nodos individuales, sino que surge de la estructura global de interacciones.

Punto 2: Funciones Sucesor, Suma y Multiplicación en Python

Base Matemática

La función sucesor $S : \mathbb{N} \to \mathbb{N}$ es la operación primitiva fundamental de la aritmética de Peano. Toda operación aritmética sobre los naturales puede derivarse únicamente a partir de ella:

$$S(n) = n + 1$$

La **suma** se define de forma recursiva como la aplicación iterada del sucesor:

$$\text{suma}(a, 0) = a \quad \text{suma}(a, b) = S(\text{suma}(a,$$

La **multiplicación** se reduce a una suma iterada:

$$\text{mult}(a, 0) = 0 \quad \text{mult}(a, b) = \text{suma}(a,$$

Implementación

El archivo `punto2_sucesor.py` contiene las tres funciones (`S`, `suma`, `multiplicacion`) construidas estrictamente sobre la función sucesor, sin usar los operadores aritméticos de Python para la suma entre naturales.

Ejecutar:

```
python punto2_sucesor.py
```

Traza de Ejecución Esperada

```
=== Función Sucesor ===
S(0) = 1
S(4) = 5

=== Suma con Sucesor ===
suma(3, 4)  = 7
suma(0, 7)  = 7
suma(10, 5) = 15

=== Multiplicación con Suma ===
multiplicacion(3, 4) = 12
multiplicacion(5, 6) = 30
multiplicacion(7, 0) = 0
```

Punto 3: Concepto de Interacción mediante Net Interactions y Programación Funcional

Base Matemática: Modelo de Net Interactions

En la teoría de sistemas complejos, una **interacción** entre entidades se modela formalmente como una función que mapea estados hacia nuevos estados. Dado un conjunto de entidades $E = \{e_1, e_2, \ldots, e_n\}$, cada una con un estado $x_{e_i}(t) \in S$, se define una función de interacción binaria:

$$I : S \times S \to \Delta S$$

donde ΔS representa el cambio de estado producido por la interacción. La **interacción neta** (*net interaction*) sobre la entidad e_i se define como la agregación de todas las interacciones que inciden sobre ella:

$$\mathrm{NI}(e_i, t) = ¿\, j \neq i \; I(x_{e_j}(t),$$

donde es un operador de agregación (suma, composición, unión) dependiente del dominio. La dinámica del sistema queda entonces descrita por:

$$x_{e_i}(t+1) = x_{e_i}(t) + \mathrm{NI}(e_i, t)$$

Esta formulación permite que el **comportamiento global emerja exclusivamente de interacciones locales**.

Programación Funcional como Herramienta

La programación funcional es idónea para modelar interacciones porque trata las funciones como ciudadanos de primera clase, evita el estado mutable y favorece la composición de funciones puras. La operación se implementa como un `reduce` (fold) sobre la lista de emisores, y la dinámica global se expresa con `map` — sin bucles imperativos ni variables compartidas.

Implementación

El archivo punto3_net_interactions.py instancia el modelo de net interactions en una aplicación real de tipo cliente–servidor. Se definen N clientes y M servidores: cada servidor es un emisor que ejerce sobre cada cliente una señal de latencia I(s, c) = (carga_s / capacidad_s)². La interacción neta NI(c) es el vector de latencias percibidas por el cliente desde todos los servidores, y su regla local es migrar al servidor de menor señal. La dinámica se implementa con map y reduce (barrido Gauss–Seidel asíncrono), sin bucles mutables, fiel al paradigma funcional.

Ejecutar:

```
python punto3_net_interactions.py
```

Traza de Ejecución Esperada

La simulación parte de un escenario patológico en el que todos los clientes apuntan al mismo servidor sobrecargado y muestra cómo, tras pocas iteraciones, la carga se redistribuye proporcionalmente a la capacidad de cada servidor. El equilibrio final emerge de interacciones locales (cada cliente sólo compara las latencias percibidas) sin intervención de un planificador central.

Punto 4: Universalidad δ como Constante de la Naturaleza y su Aplicación en Ingeniería

La Constante de Feigenbaum δ

La constante de Feigenbaum δ es una constante universal descubierta por Mitchell Feigenbaum en 1975 al estudiar la ruta al caos en sistemas dinámicos no lineales. Se define como el límite de la razón entre intervalos sucesivos de bifurcación en un diagrama de doblamiento de periodo:

$$\delta = \frac{\lim_{n \to \infty} \mu_n - \mu_{n-1}}{\mu_{n+1} - \mu_n} \approx 4.669\ \ 201\ \ 609\ \ 102\ \ 990\ldots$$

donde μ_n es el valor del parámetro de control en la n-ésima bifurcación. Esta constante aparece en el **mapa logístico**:

$$x_{n+1} = \lambda \cdot x_n (1 - x_n)$$

cuyo comportamiento depende del parámetro λ:

Rango de λ	Comportamiento del sistema
$0 < \lambda \leq 3$	Punto fijo estable (convergencia)
$3 < \lambda \leq 3.57$	Bifurcaciones de periodo doblante (2, 4, 8, ...)

$\lambda \approx 3.57$	Inicio del caos
$3.57 < \lambda \leq 4$	Comportamiento caótico

Por qué es Universal

Lo extraordinario de δ es que **es independiente de la función específica** del sistema. Aparece en cualquier sistema dinámico que mapee un intervalo sobre sí mismo y tenga un único máximo cuadrático. Esta propiedad ha sido verificada en fluidos en régimen turbulento, circuitos electrónicos no lineales, poblaciones biológicas y señales cardíacas. Es por eso que δ se considera una **constante de la naturaleza**, análoga a π o e.

Modelo Computacional: Predicción del Umbral de Caos

Si se conocen los valores del parámetro en las primeras n bifurcaciones $\mu_1, \mu_2, \ldots, \mu_n$, se puede **extrapolar** el punto de inicio del caos μ_∞ mediante:

$$\mu_\infty = \mu_n + \frac{\mu_n - \mu_{n-1}}{\delta - 1}$$

En ingeniería, esto permite determinar la **ganancia máxima segura** de un controlador antes de que el sistema entre en caos, sin necesidad de llevarlo hasta ese límite experimentalmente.

Implementación

δEl archivo punto4_feigenbaum.py aplica la universalidad δ a un problema de ingeniería de software: un algoritmo iterativo cuyo ratio de mejora de la complejidad computacional entre pasos consecutivos se modela como x_{n+1} = λ·x_n·(1 - x_n). Se fija una tolerancia constante Δ que el algoritmo debe satisfacer en cada paso (|x_{n+1} - x_n| < Δ) para declararse convergido. El script detecta las bifurcaciones sucesivas con un filtro anti-ruido (varias confirmaciones consecutivas), calcula los δ empíricos para verificar la universalidad, y predice el umbral de caos λ_∞ ≈ λ_n + (λ_n - λ_{n-1})/(δ-1). Finalmente clasifica el algoritmo bajo distintas ganancias (convergido / periódico / caótico) y recomienda una ganancia máxima segura.

Ejecutar:

python punto4_feigenbaum.py

Traza de Ejecución Esperada

Detectando bifurcaciones (ganancias críticas del algoritmo)...
λ_1 ≈ 3.00010 (período 2)
λ_2 ≈ 3.44956 (período 4)
λ_3 ≈ 3.54408 (período 8)
λ_4 ≈ 3.56456 (período 16)

δ empíricos: [4.76, 4.62]
δ Feigenbaum teórico: 4.6692016091

Umbral de caos predicho: λ_∞ ≈ 3.5701
Valor aceptado: λ_∞ ≈ 3.5699

Recomendación: operar con λ ≤ 3.5501 (umbral menos margen 0.02).

Punto 5: Modelamiento Basado en Agentes para el Concepto de Atractor

Concepto de Atractor en Sistemas Complejos

Un **atractor** es el conjunto de estados hacia el cual tiende la trayectoria de un sistema dinámico tras un periodo transitorio, independientemente de las condiciones iniciales (dentro de una cuenca de atracción). Matemáticamente, para un sistema dinámico discreto $x(t+1)=F(x(t))$, un conjunto $A \subset S$ es un atractor si:

$$\exists\ \varepsilon>0 \text{ tal que } \forall\ x(0) \in B(A,\varepsilon): \lim_{t\to\infty} d(x(t),A)=0$$

donde $B(A,\varepsilon)$ es la bola de radio ε alrededor de A (la **cuenca de atracción**). Los tres tipos principales son:

Tipo de Atractor	Descripción	Ejemplo

Punto fijo	Sistema converge a un único estado estable	Péndulo amortiguado
Ciclo límite	Sistema oscila periódicamente	Ritmo cardíaco
Atractor extraño	Dinámica caótica, sensible a condiciones iniciales	Sistema de Lorenz

Modelo Basado en Agentes (ABM) — Modelo de Deffuant

$[0,1]\varepsilon=0.5\mu=0.3$El archivo punto5_abm_atractor.py implementa una población de agentes con opiniones en [0, 1]. A diferencia de una versión homogénea, cada agente tiene SU PROPIO umbral de confianza y SU PROPIO factor μ, muestreados aleatoriamente en [0.1, 0.5]. En cada paso se selecciona al azar un par de agentes y, si ambos se aceptan mutuamente (la diferencia de opiniones es menor que ambos umbrales), cada uno se acerca al otro usando su propio μ. El script barre cuatro tamaños de población N ∈ {2, 3, 20, 50} y reporta, para cada corrida, las opiniones finales, la varianza y el número de clusters estables.

El tipo de atractor que emerge depende de la interacción entre el tamaño de la población N y la distribución heterogénea de umbrales y μ:

- • Cuando la heterogeneidad es baja y N pequeño (2–3 agentes), el sistema tiende a un atractor de punto fijo: consenso local o cuasi-consenso dentro del grupo que logra interactuar.
- • Cuando N crece (20, 50 agentes) y la heterogeneidad de umbral/μ es alta, emerge un atractor de agrupamiento: varios clusters de opinión co-existen de forma estable, porque entre clusters la distancia supera el umbral de confianza y no interactúan.

Ejecutar:

```
python punto5_abm_atractor.py
```

Emergencia del Atractor desde Reglas Locales

La característica fundamental es que el atractor **no está codificado explícitamente** en ningún agente. El comportamiento macro surge únicamente de interacciones micro, y distintas condiciones iniciales dentro de la misma cuenca de atracción llevan siempre al mismo estado final — esto ejemplifica la **emergencia** como propiedad central de los sistemas complejos.

Referencias

- Barabási, A. L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
- Feigenbaum, M. J. (1978). Quantitative universality for a class of nonlinear transformations. *Journal of Statistical Physics*, 19(1), 25–52.
- Deffuant, G., Neau, D., Amblard, F., & Weisbuch, G. (2000). Mixing beliefs among interacting agents. *Advances in Complex Systems*, 3(01–04), 87–98.
- Strogatz, S. H. (2018). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering* (2nd ed.). CRC Press.
- Tesfatsion, L. (2006). Agent-based computational economics: A constructive approach to economic theory. *Handbook of Computational Economics*, 2, 831–880.
- Giordano, P. (2024). Interaction spaces: a universal theory of complex systems. *arXiv preprint arXiv:2407.02175*.